

Name - Chaitanya Kelkar

Roll no - 15

Experiment No 5

Aim

Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory

1. Primitive Data Types, Variables, Functions (pure, view)

- **Primitive Data Types:** Solidity includes basic data types such as uint/int (unsigned and signed integers of varying sizes like uint256 and int128), bool (boolean values true or false), address (20-byte Ethereum account or contract address), and bytes/string (for binary and textual data).
- **Variables:** Consist of state variables (permanently stored on the blockchain), local variables (temporary variables created within function execution), and global variables (predefined variables providing blockchain contextual data such as msg.sender, msg.value, and block.timestamp).
- **Functions (pure, view):** pure functions neither read from nor modify the blockchain state. view functions read state variables but cannot modify them. These modifiers assist in optimizing gas consumption and enforcing contract integrity.

2. Inputs and Outputs to Functions

- Functions accept input arguments to pass data into the contract and return one or more output values after logic execution.
- Solidity supports named return variables, enhancing code readability and debugging efficiency.

3. Visibility, Modifiers, and Constructors

- **Visibility:** Defines access permissions for functions and variables:
 - public: Accessible both internally within the contract and externally.
 - private: Accessible only within the contract where defined.
 - internal: Accessible within the contract and derived/child contracts.
 - external: Only callable by external accounts or contracts.
- **Modifiers:** Reusable blocks of code that alter function behavior, commonly utilized for access control mechanisms like restricting function execution to a contract owner.
- **Constructors:** Special initialization functions executed only once upon contract deployment to set initial state values.

4. Control Flow: if-else and Loops

- **if-else:** Enables conditional decision-making in contract logic (e.g., verifying user balances before performing transfers).

- **Loops:** Standard looping constructs (for, while, do-while) allow repeated code execution. Iteration must be managed carefully as excessive loop execution increases gas consumption, leading to potential transaction failures.

5. Data Structures: Arrays, Mappings, Structs, Enums

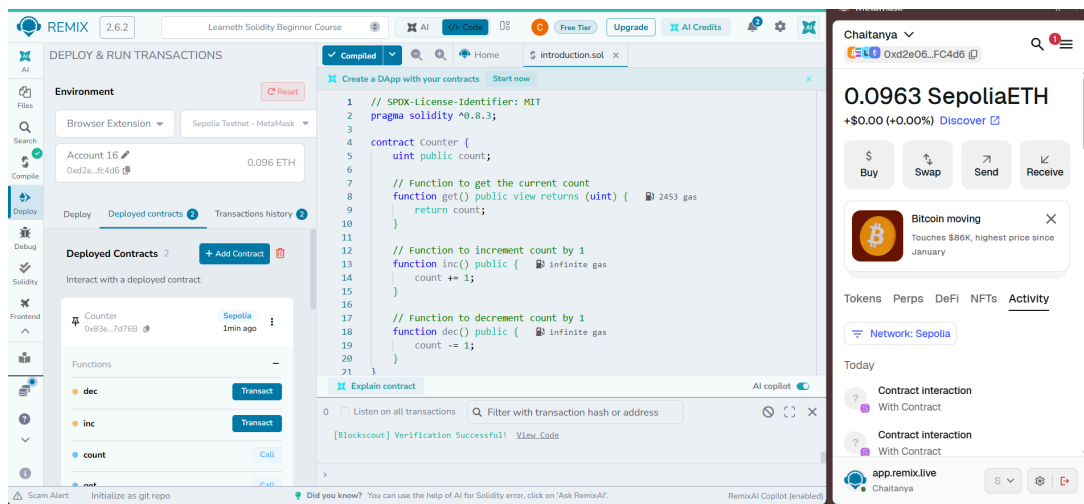
- **Arrays:** Ordered collections of elements that can be fixed or dynamic in size.
- **Mappings:** Key-value stores providing efficient lookups (e.g., mapping(address => uint)), though they do not support native iteration.
- **Structs:** Custom user-defined types that group related variables into a single structure.
- **Enums:** User-defined types consisting of explicit set of predefined constants to improve code readability.

6. Data Locations

- **storage:** Permanent data location stored on the blockchain (used for state variables).
- **memory:** Temporary data storage that exists only during function execution.
- **calldata:** Non-modifiable, non-persistent data location used for external function parameters, offering higher gas efficiency than memory.

7. Transactions: Ether & Wei, Gas & Gas Price, Sending Transactions

- **Ether and Wei:** Ether is Ethereum's native cryptocurrency, measured internally in Wei, the smallest unit ($1 \text{ Ether} = 10^{18} \text{ Wei}$), ensuring precision in financial transactions.
- **Gas and Gas Price:** Gas measures the computational work required to execute transactions. Gas price determines the Ether paid per gas unit, incentivizing validators to process transactions.
- **Sending Transactions:** Used to transfer Ether or trigger contract functions using mechanisms like transfer(), send(), or call().



REMIX2.6.2Learneth Solidity Beginner CourseAI CodeHomeintroduction.solbasicSyntax.solx

LEARNETH

Tutorials listSyllabus

2. Basic Syntax2 / 19

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

Watch a video tutorial on Basic Syntax.

Assignment

1. Delete the HelloWorld contract and its content.

2. Create a new contract named "MyContract".

3. The contract should have a public state variable called "name" of the type string.

4. Assign the value "Alice" to your new variable.

Check Answer

Show answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract MyContract {
5     string public name = "Alice";
6 }
```

Explain contractAI copilot

☐ Listen on all transactions

Filter with transaction hash or address

[Blockscout] Verification Successful!View Code

REMIX2.6.2Learneth Solidity Beginner CourseAI CodeHomeintroduction.solbasicSyntax.solprimitiveDataTypes.solx

LEARNETH

Tutorials listSyllabus

3. Primitive Data Types3 / 19

Later in the course, we will look at data structures like **Mappings**, **Arrays**, **Enums**, and **Structs**.

Watch a video tutorial on Primitive Data Types.

Assignment

1. Create a new variable `newAddr` that is a `public` `address` and give it a value that is not the same as the available variable `addr`.

2. Create a `public` variable called `neg` that is a negative number, decide upon the type.

3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer

Show answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Primitives {
5
6     // 1. A public address variable with a different value
7     address public newAddr = 0x58380a6a701c568545dcfc803fc8875f56beddc4;
8
9     // 2. A public variable for a negative number (using int, which supports negative values)
10    int public neg = -10;
11
12    // 3. The smallest uint size type (uint8) and its smallest possible value (0)
13    uint8 public newU = 0;
14
15    bool public boo = true;
16
17    /*
18     * uint stands for unsigned integer, meaning non negative integers
19     * different sizes are available
20     * uint8 ranges from 0 to 2 ** 8 - 1
21     * uint16 ranges from 0 to 2 ** 16 - 1
22     */
23 }
```

Explain contractAI copilot

☐ Listen on all transactions

Filter with transaction hash or address

[Blockscout] Verification Successful!View Code

REMIX2.6.2Learneth Solidity Beginner CourseAI CodeDAppintroduction.solbasicSyntax.solprimitiveDataTypes.solvariables.sol3x

LEARNETH

Tutorials listSyllabus

4. Variables4 / 19

to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions. In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

Assignment

1. Create a new public state variable called `blockNumber`.

2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8
9     // 1. Create a new public state variable called blockNumber
10    uint public blockNumber;
11
12    function doSomething() public { 22334 gas
13        // local variables are not saved to the blockchain.
14        uint i = 456;
15
16        // Here are some global variables
17        uint timestamp = block.timestamp; // Current block timestamp
18        address sender = msg.sender; // address of the caller
19
20        // 2. Assign the value of the current block number to blockNumber
21        blockNumber = block.number;
22    }
```

Explain contractAI copilot

☐ Listen on all transactions

Filter with transaction hash or address

Type the library name to see available commands.

LEARNETH

Tutorials list

Syllabus

5.1 Functions - Reading and Writing to a State Variable

5 / 19

To define a function, use the `function` keyword followed by a unique name.

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `set` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections. Watch a video tutorial on Functions.

Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.

2. Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

basicSyntax.sol

primitiveDataTypes.sol

variables.sol 3

readAndWrite.sol

Create a DApp with your contracts

Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7
8     // 1. Create a public state variable called b of type bool initialized to true
9     bool public b = true;
10
11     // You need to send a transaction to write to a state variable.
12     function set(uint _num) public {
13         num = _num;
14     }
15
16     // You can read from a state variable without sending a transaction.
17     function get() public view returns (uint) {
18         return num;
19     }
20
21     // 2. Create a public function called get_b that returns the value of b
22 }
```

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or address

Type the library name to see available commands.

REMUX 2.6.2

Learneth Solidity Beginner Course

AI

20 Code

Free Tier

Upgrade

Get AI Credits

LEARNETH

Tutorials list

Syllabus

5.2 Functions - View and Pure

6 / 19

5. Using inline assembly that contains certain opcodes."

From the Solidity documentation.

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `x` and `y`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

Assignment

Create a function called `addToX` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

primitiveDataTypes.sol

variables.sol 3

readAndWrite.sol

viewAndPure.sol

Create a DApp with your contracts

Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     function addToX(uint y) public {
8         x = x + y;
9     }
10
11     // Promise not to modify the state.
12     function addToX(uint y) public view returns (uint) {
13         return x + y;
14     }
15
16     // Promise not to modify or read from the state.
17     function add(uint i, uint j) public pure returns (uint) {
18         return i + j;
19     }
20 }
```

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or address

Type the library name to see available commands.

REMUX 2.6.2

Learneth Solidity Beginner Course

AI

20 Code

Free Tier

Upgrade

Get AI Credits

LEARNETH

Tutorials list

Syllabus

5.3 Functions - Modifiers and Constructors

7 / 19

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.

2. Make sure that `x` can only be increased.

3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

Compile

variables.sol 3

readAndWrite.sol

viewAndPure.sol

modifiersAndConstructors.sol

Create a DApp with your contracts

Start now

```
50
51 if (i > 1) {
52     decrement(i - 1);
53 }
54
55 // 1. Create a modifier that takes the input and increases x
56 modifier onlyIncrease(uint _val) {
57     // A uint cannot be negative, so adding it ensures x only increases
58     x += _val;
59     _;
60 }
61
62 // 2. Create the increaseX function, apply the modifier, and leave the body empty
63 function increaseX(uint _val) public onlyIncrease(_val) {
64
65 }
```

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or address

Type the library name to see available commands.

REMIX2.6.2Learneth Solidity Beginner Course

AI</> Code

Free TierUpgradeGet AI Credits

LEARNETH

Tutorials listSyllabus

5.3 Functions - Modifiers and Constructors7 / 19

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.
You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.
[Watch a video tutorial on Function Modifiers.](#)

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.
Tip: Use modifiers.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

50515253545556575859606162636465

```
if (i > 1) {
    decrement(i - 1);
}

// 1. Create a modifier that takes the input and increases x
modifier onlyIncrease(uint _val) {
    // A uint cannot be negative, so adding it ensures x only increases
    x += _val;
}

// 2. Create the increaseX function, apply the modifier, and leave the body empty
function increaseX(uint _val) public onlyIncrease(_val) {
    infinite gas
}
```

Explain contractAI copilot

0Listen on all transactionsFilter with transaction hash or address

Type the library name to see available commands.

REMIX2.6.2Learneth Solidity Beginner Course

AI</> Code

Free TierUpgradeGet AI Credits

LEARNETH

Tutorials listSyllabus

5.4 Functions - Inputs and Outputs8 / 19

There are a few restrictions and best practices for the input and output parameters of contract functions.
"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the [Solidity documentation](#).
Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).
You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.
[Watch a video tutorial on Function Outputs.](#)

Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

7273747576777879808182838485

```
// Can use array for output
uint[] public arr;

function arrayOutput() public view returns (uint[] memory) {
    return arr;
}

// Returns -2 and true without a return statement using named returns
function returnTwo() public pure returns (int i, bool b) {
    i = -2;
    b = true;
}
```

Explain contractAI copilot

0Listen on all transactionsFilter with transaction hash or address

Type the library name to see available commands.

REMIX2.6.2Learneth Solidity Beginner Course

AI</> Code

Free TierUpgradeGet AI Credits

LEARNETH

Tutorials listSyllabus

6. Visibility9 / 19

- Can be called from other contracts or transactions
- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.
When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.
If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.
[Watch a video tutorial on Visibility.](#)

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

47484950515253545556575859606162636465666768

```
// State variables
string private privateVar = "my private variable";
string internal internalVar = "my internal variable";
string public publicVar = "my public variable";

// State variables cannot be external so this code won't compile.
// string external externalVar = "my external variable";

contract Child is Base {
    // Inherited contracts do not have access to private functions
    // and state variables.
    // function testPrivateFunc() public pure returns (string memory) {
    //     return privateFunc();
    // }
    function testInternalVar() public view returns (string memory, string memory) {
        return (internalVar, publicVar);
    }

    // Internal function call be called inside child contracts.
    function testInternalFunc() public pure override returns (string memory) {
        return internalFunc();
    }
}
```

Explain contractAI copilot

0Listen on all transactionsFilter with transaction hash or address

Type the library name to see available commands.

REMIX2.6.2Learneth Solidity Beginner Course

LEARNETH

Tutorials listSyllabus

7.1 Control Flow - If/Else10 / 19

the other conditions are met.

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the `If/Else` statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check AnswerShow answer

Next

Well done! No errors.

CompileCode

modifiersAndConstructors.solinputsAndOutputs.solvisibility.solIfElse.sol

Create a DApp with your contractsStart now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract IfElse {
5     function evenCheck(uint _num) public pure returns (bool) {
6         return _num % 2 == 0 ? true : false;
7     }
8
9     function foo(uint x) public pure returns (uint) {
10         if (x < 10) {
11             return 0;
12         } else if (x < 20) {
13             return 1;
14         } else {
15             return 2;
16         }
17     }
18
19     function ternary(uint _x) public pure returns (uint) {
20         // if (_x < 10) {
21             return 1;
22         }
23     }
24 }
```

Explain contract

AI copilot

0 Listen on all transactionsFilter with transaction hash or address

Type the library name to see available commands.

REMIX2.6.2Learneth Solidity Beginner Course

LEARNETH

Tutorials listSyllabus

7.2 Control Flow - Loops11 / 19

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check AnswerShow answer

Next

Well done! No errors.

CompileCode

AndConstructors.solinputsAndOutputs.solvisibility.solIfElse.solloops.sol

Create a DApp with your contractsStart now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Loop {
5     // 1. Create a public uint state variable called count
6     uint public count;
7
8     function loop() public {
9         // for loop
10         for (uint i = 0; i < 10; i++) {
11             if (i == 3) {
12                 // skip to next iteration with continue
13                 continue;
14             }
15
16             // 3. Change the condition so the break statement isn't triggered early
17             if (i == 10) {
18                 // Exit loop with break
19                 break;
20             }
21
22             // 2. Increment the count variable by 1 at the end of the for loop.
23             count++;
24         }
25     }
26 }
```

Explain contract

AI copilot

0 Listen on all transactionsFilter with transaction hash or address

Type the library name to see available commands.

REMIX2.6.2Learneth Solidity Beginner Course

LEARNETH

Tutorials listSyllabus

8.1 Data Structures - Arrays12 / 19

(line 42). When we remove an element with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the `length` member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

- Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
- Change the `getArr()` function to return the value of `arr3`.

Check AnswerShow answer

Next

Well done! No errors.

CompileCode

inputsAndOutputs.solvisibility.solIfElse.solloops.solarrays.sol

Create a DApp with your contractsStart now

```
25 // SPDX-License-Identifier: MIT
26 // This will increase the array length by 1.
27 arr.push(1);
28 }
29
30 function pop() public {
31     // Remove last element from array
32     // This will decrease the array length by 1
33     arr.pop();
34 }
35
36 function getLength() public view returns (uint) {
37     return arr.length;
38 }
39
40 function remove(uint index) public {
41     // Delete does not change the array length.
42     // It resets the value at index to its default value,
43     // in this case 0
44     delete arr[index];
45 }
46 }
```

Explain contract

AI copilot

0 Listen on all transactionsFilter with transaction hash or address

Type the library name to see available commands.

REMIX2.6.2Learneth Solidity Beginner Course

AIv2 Code

Free TierUpgradeGet AI Credits

LEARNETH

Tutorials listSyllabus

8.2 Data Structures - Mappings13 / 19

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.

2. Change the functions `get` and `remove` to work with the mapping balances.

3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5     // 1. Create a public mapping balances that associates address with uint
6     mapping(address -> uint) public balances;
7
8     function get(address _addr) public view returns (uint) {
9         // 2. Change get to work with balances
10         return balances[_addr];
11     }
12
13     function set(address _addr) public {
14         // 3. Set the value to the Ethereum balance of the address parameter
15         balances[_addr] = _addr.balance;
16     }
17
18     function remove(address _addr) public {
19         // 2. Change remove to work with balances
20         delete balances[_addr];
21     }
22 }
```

Explain contract

AI copilot

0 Listen on all transactionsFilter with transaction hash or address

Type the library name to see available commands.

REMIX2.6.2Learneth Solidity Beginner Course

AIv2 Code

Free TierUpgradeGet AI Credits

LEARNETH

Tutorials listSyllabus

8.3 Data Structures - Structs14 / 19

inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Todos {
5     struct Todo {
6         string text;
7         bool completed;
8     }
9
10    // An array of 'Todo' structs
11    Todo[] public todos;
12
13    function remove(uint _index) public {
14        // infinite gas
15        delete todos[_index];
16    }
17
18    function create(string memory _text) public {
19        // infinite gas
20        // 3 ways to initialize a struct
21        // - calling it like a function
22        todos.push(Todo(_text, false));
23    }
24 }
```

Explain contract

AI copilot

0 Listen on all transactionsFilter with transaction hash or address

Type the library name to see available commands.

REMIX2.6.2Learneth Solidity Beginner Course

AIv2 Code

Free TierUpgradeGet AI Credits

LEARNETH

Tutorials listSyllabus

8.4 Data Structures - Enums15 / 19

Updating an enum value

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.

2. Initialize the variable `sizes` of the enum type `Size`.

3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
4 contract Enum {
5     // Enum representing shipping status
6     enum Status {
7         Pending,
8         Shipped,
9         Accepted,
10        Rejected,
11        Canceled
12    }
13
14    // 1. Define an enum type called Size with members S, M, and L
15    enum Size { S, M, L }
16
17    // 2. Initialize the variable sizes of the enum type Size
18    Size public sizes;
19
20    // 3. Create a getter function getSize() that returns the value of the variable sizes
21    function getSize() public view returns (Size) {
22        // 2582 gas
23        return sizes;
24    }
25 }
```

Explain contract

AI copilot

0 Listen on all transactionsFilter with transaction hash or address

Type the library name to see available commands.

REMIX 2.6.2 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

9. Data Locations 16 / 19

not affect the values stored in the mapping `myStructs` (line 10).
As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer Show answer

Next

Well done! No errors.

```
11
12 function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory) {
13     // call f with state variables
14     _f(arr, map, myStructs[1]);
15
16     // get a struct from a mapping
17     MyStruct storage myStruct = myStructs[1];
18     // create a struct in memory
19     MyStruct memory myMemStruct = MyStruct(0);
20
21     // 1. Change the value of myStruct.foo to 4
22     myStruct.foo = 4;
23
24     // 2. Create myMemStruct2, assign myMemStruct, and change foo to 1
25     MyStruct memory myMemStruct2 = myMemStruct;
26     myMemStruct2.foo = 1;
27
28     // 3. Create myMemStruct3, assign myStruct, and change foo to 3
29     MyStruct memory myMemStruct3 = myStruct;
30     myMemStruct3.foo = 3;
31
32     // 4. Return all three structs
```

REMIX 2.6.2 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

10.1 Transactions - Ether and Wei 17 / 19

Wei is the smallest subunit of Ether, named after the cryptographer Wei Dai. Ether numbers without a suffix are treated as `wei` (line 7).

`gwei`
One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`
One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).
Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1. create a public uint called oneGwei and set it to 1 gwei
7     uint public oneGwei = 1 gwei;
8
9     // 2. Create a public bool called isOneGwei and compare 1 gwei to 10^9 (1e9)
10    bool public isOneGwei = 1 gwei == 1e9;
11
12    // 1 wei is equal to 1
13    bool public isOneWei = 1 wei == 1;
14
15    uint public oneEther = 1 ether;
16    // 1 ether is equal to 10^18 wei
17    bool public isOneEther = 1 ether == 1e18;
18 }
```

REMIX 2.6.2 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

10.2 Transactions - Gas and Gas Price 18 / 19

gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.
Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

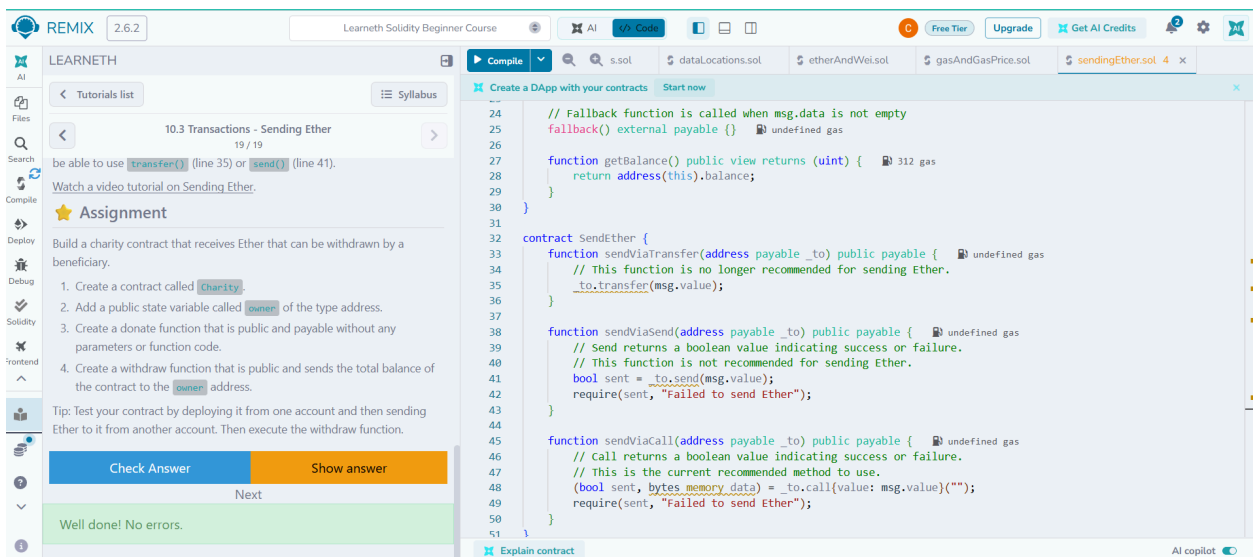
Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin `Gas Profiler` to check for the gas cost of transactions.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6
7     uint public cost = 170367;
8
9     // Using up all of the gas that you send causes your transaction to fail.
10    // State changes are undone.
11    // Gas spent are not refunded.
12    function forever() public {
13        // Here we run a loop until all of the gas are spent
14        // and the transaction fails
15        while (true) {
```



Conclusion

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.